

PD-PRUNE: Split Prefill and Decode Pruning for Efficient Agentic Inference

Anonymous ACL submission

Abstract

Large language models used as agents expand one committed trunk into several residuals and later keep a single spine. This serving pattern is inefficient: search and early-stop methods score tokens that have already been generated, so residuals that will be aborted still pay GPU prefill, KV transfer, and decode until a prune score exists. The waste stems from the stack’s inability to refuse a residual before tokens or pages exist. We propose PD-PRUNE, which reduces that waste by identifying and pruning abortable residuals during KV-cache encoding of the residual, then deciding continuation from a short generated prefix. The pruning steers the stack toward treating a loop as skippable text rather than as a thought to encode, while a separate decode score preserves residuals that can still solve once a prefix is visible. To identify what to prune, we introduce two unshared scores with ordered bars ($\tau_{\text{pre}}, \tau_{\text{dec}}$), and we connect the prefill bar to an invariance of prefill work across decode-time hosts. Our approach does not replace the host’s cut or incur extra model calls at test time. Experiments show that PD-PRUNE reduces the prefill shared by three decode-time pruners while preserving any-finished accuracy, and that a short probe restores full MATH-500 accuracy at lower end-to-end time.

1 Introduction

The rapid adoption of large language models as agents (Yao et al., 2023a,b; Wang et al., 2023) has turned inference from a single continuation into a search process. An agentic step expands a committed trunk of length L into k residuals and later keeps a single spine (Figure 1). This pattern is the serving unit of Tree-of-Thoughts, self-consistency, and tool-using loops, and it is where latency and memory are spent. However, existing stacks exhibit a critical inefficiency: residuals that the search will abort still incur GPU prefill, KV transfer, and decode until a prune score exists. That waste grows

with fan-out width and is paid on every step, posing a significant cost to the reliability of agentic inference in real-world applications.

The susceptibility of LLM serving to this waste stems from a fundamental limitation in parsing the fan-out, i.e., the inability to distinguish a residual that must be encoded from one that can be refused as text (Li et al., 2024; Sun et al., 2024; Ding et al., 2025). In reducing that cost, replacing the search algorithm, or retraining the model to emit fewer thoughts, can be computationally prohibitive and is orthogonal to serving. Alternatively, existing mitigation strategies focus on scoring tokens that have already been generated (Li et al., 2024; Sun et al., 2024; Ding et al., 2025), on recording a prefix only after a full block is hashed (Kwon et al., 2023; Zheng et al., 2024), or on transferring every page a disaggregated prefiller has computed (Zhong et al., 2024; Patel et al., 2024). Such strategies often provide only decode-side savings, or they ship KV that the search is about to drop. Additionally, imposing a single threshold on both prefill and decode can drop residuals that still solve, thereby undermining the quality of the kept spine.

In this paper, we focus on the source of that inefficiency: the stack’s confusion between a residual that must be prefilled and a residual that can be skipped. We start from a fundamental question: *what makes the difference between a residual that should be encoded and one that should not, from the serving stack’s perspective?* The engine relies on implicit criteria that only become observable after tokens exist. Concretely, from a cost perspective, a residual is *live* if some continuation of it can still produce a correct answer, and *abortable* if it is a repeated loop or illegal text that the search will drop. Waste arises when those implicit criteria fail to align with the live–abortable separation that is already present in the residual text. Our approach is motivated by solving that misalignment with the following two general steps.

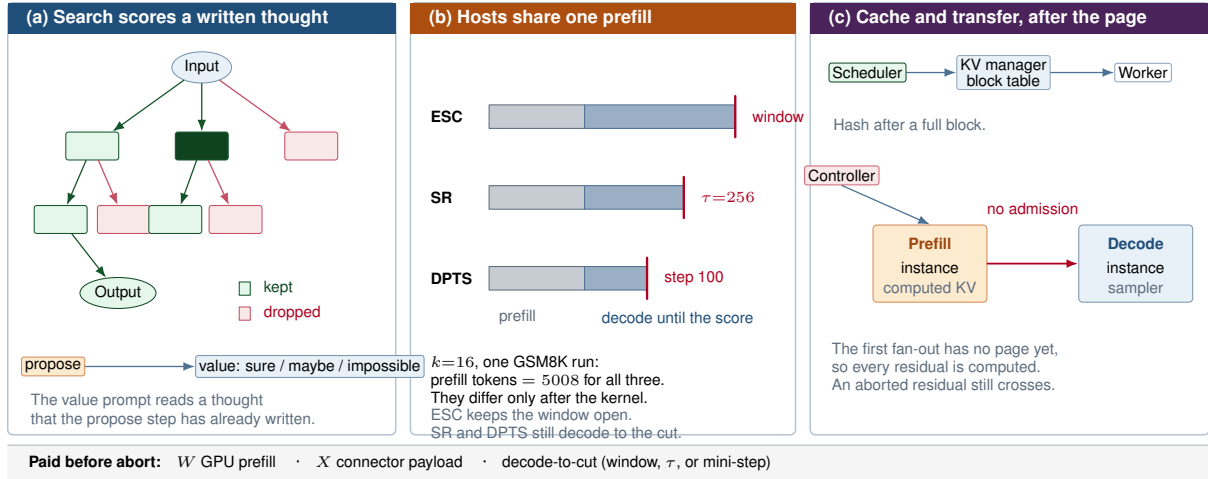


Figure 1: Illustration of wasted work in an agentic fan-out. (a) Tree-of-Thoughts writes a thought and then values it (Yao et al., 2023a). (b) ESC, speculative rejection, and DPTS read a prefix that is already resident (Li et al., 2024; Sun et al., 2024; Ding et al., 2025); on this GSM8K run their prefill token count is the same. (c) Automatic prefix caching hashes a full block (Kwon et al., 2023), and a disaggregated runtime transfers the KV the prefiller has already computed (Zhong et al., 2024). GPU prefill W , connector payload X , and decode-to-cut are incurred before the abort.

- **Prefill scoring.** We identify abortable residuals by a score on the residual text, whose activations bias the stack toward encoding the same span as a thought rather than as skip-pable text.
- **Decode intervention.** We prune the identified residuals over the prefill of the fan-out, and we continue an uncertain residual only when a short generated prefix clears a second bar.

Specifically, we introduce two unshared scores with ordered bars ($\tau_{\text{pre}}, \tau_{\text{dec}}$), $\tau_{\text{pre}} < \tau_{\text{dec}}$. The prefill score reads the residual and drops a repeated loop before the kernel and before any KV transfer. The decode score reads a generated prefix after a probe of t_0 tokens and continues to the budget D only when that prefix clears τ_{dec} . We impose the invariant that index 0 is always admitted, so pruning cannot erase the residual the search will keep if nothing else survives. We show that the proposed split is host-invariant in prefill work: any decode-time pruner that scores only generated tokens shares one W , and that W is exactly the prefill of the admitted set.

For compatibility with prefix caching and prefill-decode disaggregation (Kwon et al., 2023; Zhong et al., 2024; Patel et al., 2024), we apply pruning when encoding the KV cache of the residual, i.e., enabling efficient prompting when several residuals share the same trunk. We accordingly refer to our approach as PD-PRUNE. Notably, PD-PRUNE

leaves the host’s decode cut unchanged and is lightweight in computation, relying solely on a pair of string scores without extra test-time model calls.

Our contributions are summarized as follows:

- We propose PD-PRUNE, which reduces wasted prefill in agentic inference by identifying and pruning abortable residuals during KV-cache encoding of the residual. It steers the stack toward treating a loop as skippable text, while not interfering with the model’s ability to follow live strategies.
- In identifying these residuals, we propose a split scoring mechanism that allows effective pruning from surface tests on the residual and on a short prefix. We also leverage an observed score gap—illegal text lies above τ_{pre} on the residual and below τ_{dec} on a still-illegal prefix—that further improves the fidelity of the split.
- In experiments, PD-PRUNE significantly reduces the prefill shared by ESC, speculative rejection, and DPTS while preserving any-finished accuracy, and a short probe restores full MATH-500 accuracy at lower end-to-end time.

2 PD-PRUNE

2.1 Preliminary

Agentic prompting. Let $x = [x_t]_{t=1}^L$ denote a committed trunk of length L , and let $\rho_0, \dots, \rho_{k-1}$ denote k residuals proposed from x . Each candidate sequence is $z_i = x \circ \rho_i$. After the step, a single spine $x \circ \rho_*$ remains live. $p_\theta(\cdot \mid z_i)$ is the next-token distribution of an LLM parameterized by θ . The model is expected to answer from at least one live residual, while abortable residuals should contribute neither prefill nor transfer.

State-of-the-art LLMs adopt the Transformer architecture (Vaswani et al., 2017), where each token of z_i is encoded by layer ℓ into a key and a value. Let H_i be the KV cache of z_i after prefill. For a continuation $y_i = [y_{i,t}]_{t=1}^D$ of budget D ,

$$p_\theta(y_{i,t} \mid z_i, y_{i,<t}) = p(y_{i,t} \mid H_i, y_{i,<t}, \theta). \quad (1)$$

H_i is reused during decode with different $y_{i,t}$. On a copy-on-write tree, x is aliased in $O(1)$ and only ρ_i allocates new pages; live occupancy after abort is the spine.

Costs of one fan-out. The interval that matters is from the moment the residuals are proposed until one of them is committed and the rest are aborted. That interval contains three costs. GPU prefill W is the tokens the kernel actually runs. The connector payload X is the KV that a disaggregated prefill instance ships to a decode instance (Zhong et al., 2024; Patel et al., 2024). Decode-to-cut is the prefix that must be generated before a prune score exists. On the systems in Figure 1, all three are paid before the abort.

Decode-time pruning. Early-stopping self-consistency (ESC), speculative rejection, and dynamic parallel tree search (DPTS) score tokens that are already resident (Li et al., 2024; Sun et al., 2024; Ding et al., 2025). As illustrated in Figure 1(b), we define y_i as a continuation of z_i under a host cut: ESC decodes to D while an agreement window is open, speculative rejection scores at a partial-reward horizon, and DPTS generates a mini-step. A residual is wasted if H_i is computed and y_i is later aborted. An LLM serving stack is inefficient on the fan-out if abortable residuals are preferred for encoding over skipping, i.e., if they enter the prefill batch.

2.2 Refusing Abortable Residuals

We refuse by steering the stack to interpret abortable residuals as skippable text rather than as thoughts to encode, thereby reducing W and X in response generation. To achieve this, we propose PD-PRUNE in two stages, Prefill Admission and Decode Continuation, illustrated in Figure 2.

2.2.1 Prefill Admission

As mentioned in Section 1, we aim at identifying residuals whose text biases the stack toward encoding the same span as a thought rather than as skippable text. These residuals should not interfere with the model’s ability to follow live strategies, i.e., by generating accurate answers that leverage the trunk as supporting context. The identified residuals are excluded from the KV cache through the following steps.

Scoring. Suppose there exists a prefill score $s^{\text{pre}} : \rho \mapsto [0, 1]$ that captures the stack’s preference over which residuals to encode, i.e., by measuring how much a residual looks like a live strategy rather than a loop. Residuals scoring below τ_{pre} can be viewed as steering the kernel toward wasted prefill. We score only the residual span ρ_i , not the trunk x , because abortable text is proposed in the residual. Details of s^{pre} are in Section 2.3.

Forced keep. Index 0 is the residual the search will keep if nothing else survives. Pruning it would degrade the spine even when every sibling is abortable. We therefore admit

$$\text{keep}_i = [i = 0 \vee s^{\text{pre}}(\rho_i) \geq \tau_{\text{pre}}]. \quad (2)$$

Only residuals with keep_i are queued for the kernel. W is the measured prefill of that set. With prefix caching on, a shared trunk is not the dominant term; the gate’s effect is that a dropped residual contributes nothing. The connector inserts KV only for the admitted set, so a dropped residual contributes $X_i = 0$ as well.

Thresholding. Pruning residuals with small s^{pre} should alter the stack’s tendency to encode abortable text. However, this would degrade live answers if s^{pre} were reused as a decode bar, because that score does not observe a generated prefix. To maintain quality after pruning, we regularize by a second bar $\tau_{\text{dec}} > \tau_{\text{pre}}$ that is never applied to residual text in the default policy. Raising the prefill bar to τ_{dec} is a faster operating point that we measure, not the default.

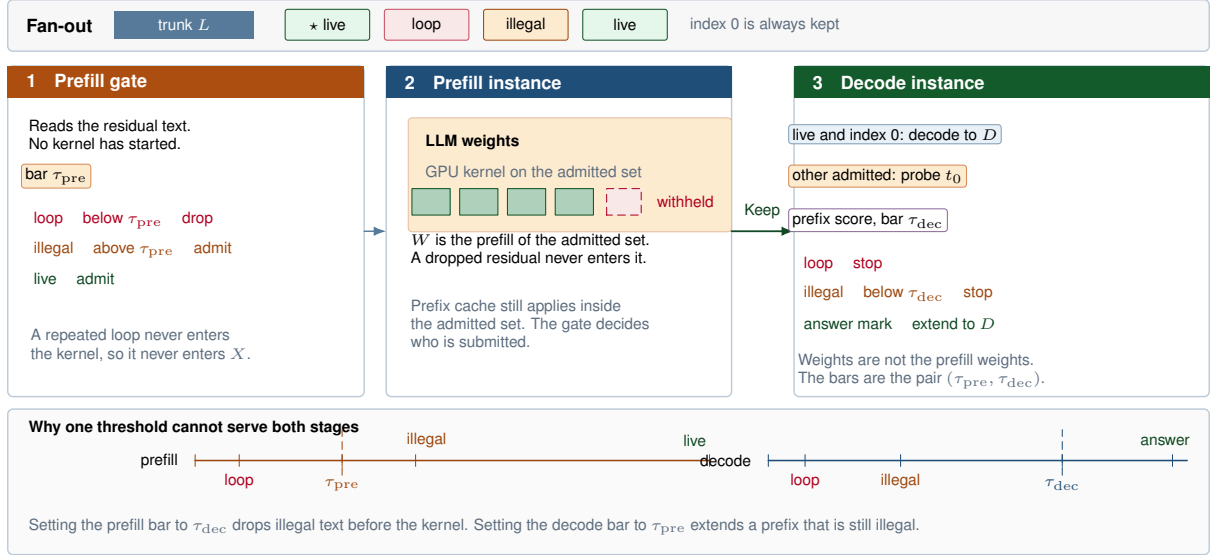


Figure 2: Illustration of the workflow of PD-PRUNE. Given a fan-out with live strategies, a loop, and illegal text, prefill scoring is computed from residual text before any kernel. A residual is pruned by withholding it from the context KV cache (orange). Activations after admission are generated only on the admitted cache. Continuation uses a separate score on a generated prefix of length t_0 (green). The number lines show why the two bars are not interchangeable: illegal text sits above τ_{pre} and a still-illegal prefix sits below τ_{dec} .

2.2.2 Decode Continuation

In experiments, the prefill mask is computed from residual text with no model call, then applied to every fan-out at test time. Admitted residuals that the decode stage later stops have already paid their prefill; they do not pay the rest of D .

Decode length is chosen after admission. A repeated loop is not probed: it never starts. Index 0 and any residual whose planner prior is already at least τ_{dec} decode to the budget D . Every other admitted residual decodes t_0 tokens, and is extended to D only when a decode score s^{dec} on the generated prefix clears τ_{dec} :

$$\text{extend}_i = [s^{dec}(y_{i,1:t_0}) \geq \tau_{dec}]. \quad (3)$$

Algorithm 1 is this rule. Section 2.5 is the special case in which a decode-time host, rather than t_0 , supplies the later cut.

Applying this mask over the residual ensures that the stack’s implicit criteria align with the live–abortable separation already present in the text. Varying the prefill bar, as in Section 4, shows a trade-off between robustness of any-finished accuracy and quality of following only the winner.

2.3 The Two Scores

As described in Section 2.2, admission is guided by s^{pre} and continuation by s^{dec} . Both are deterministic functions of a string. They do not call the model. They do not share weights.

Algorithm 1 Probe, then extend. Loops skip prefill. Index 0 decodes to D .

Require: residuals $\rho_{0:k-1}$, budget D , probe t_0 , bars (τ_{pre}, τ_{dec})

```

1: for  $i \leftarrow 0$  to  $k - 1$  do
2:   if  $i \neq 0$  and  $\rho_i$  is a repeated loop then
3:     skip ▷ no kernel,  $X_i=0$ 
4:   else if  $i = 0$  or planner prior of  $\rho_i \geq \tau_{dec}$  then
5:     prefill  $\rho_i$ ; decode to  $D$ 
6:   else
7:     prefill  $\rho_i$ ; decode  $t_0$  tokens
8:     if  $s^{dec}(y_{i,1:t_0}) \geq \tau_{dec}$  then
9:       extend from  $t_0$  to  $D$ 
10:    end if
11:  end if
12: end for

```

The prefill scorer looks only at ρ_i . A repeated span is a loop and falls below τ_{pre} . A residual that matches an illegal-math pattern, such as a run of stars or an undefined token, lands strictly between τ_{pre} and τ_{dec} . Any other non-empty residual receives

$$s^{pre}(\rho_i) = 0.55 \min(1, 2\alpha_i) + 0.45 \min(1, 8\eta_i), \quad (4)$$

where α_i is the alphanumeric fraction and η_i is the character-level uniqueness. Live strategy text clears both bars.

The decode scorer looks only at a generated prefix y_i . A repeated prefix stays below τ_{dec} . An answer mark (#### or \boxed) clears τ_{dec} . Illegal text on a prefix also stays below τ_{dec} , which is why it cannot share τ_{pre} . Otherwise

$$s^{\text{dec}}(y_i) = 0.50 \min(1, 2\alpha) + 0.30 \min(1, 8\eta) + 0.20 \min(1, 20\delta + 15\omega), \quad (5)$$

with δ the digit fraction and ω the fraction of arithmetic symbols. A static planner prior used by the host-side ranker is a third mix again (0.35 alphanumeric, 0.65 uniqueness) and is never reused as the prefill score.

The operating point is the ordered pair $(\tau_{\text{pre}}, \tau_{\text{dec}})$ with $\tau_{\text{pre}} < \tau_{\text{dec}}$ (Equation 6 in the appendix gives the run). On the prefill axis the loop is below τ_{pre} and illegal text is above it, so the gate drops loops and keeps illegal text. On the decode axis illegal text is below τ_{dec} and an answer mark is above it, so a probe stops on illegal text and continues once an answer mark appears. Section 4 measures what happens when a single bar is used for both decisions.

The subset that must survive. In Section 2.2.1, we regularize admission by forcing index 0 and by not using τ_{dec} as the default prefill bar. We want to avoid pruning residuals whose prefill score is that of live strategy text, so that the pruned stack can still generate answers that address the user query by leveraging the trunk. Index 0 is among the live set in our evaluation mix, so the forced keep does not add a further slot beyond the live half. A batch-limited rule that ignores the score and keeps a single residual is a different policy; Section 4 shows that it discards correct branches.

Proposition 1 (Host-invariant prefill). *Let $\text{Keep} = \{i : \text{keep}_i\}$ be the admitted set of (2). For any decode-time host that scores only generated tokens, the prefill work W equals the measured prefill of Keep and does not depend on the host.*

The proof is in Appendix C. Proposition 1 is the reason three otherwise different hosts share one prefill column in Figure 1(b) and in Table 1.

2.4 The Score Gap

Prefill scoring with (4) relies on the residual string alone. Not every abortable residual is a loop. Specifically, we find that the same illegal span can be interpreted as a thought to encode or as a prefix to stop, depending on whether the score is read

before or after generation. We refer to this phenomenon as the *score gap*.

To illustrate, illegal text scores above τ_{pre} on the residual and below τ_{dec} on a generated prefix that is still illegal. A loop is below both bars. A live residual clears τ_{pre} ; a prefix that has emitted an answer mark clears τ_{dec} . Figure 2 plots these four points on two number lines. The gap is why a single bar is the wrong cut: applying τ_{dec} as a prefill bar drops illegal residuals that can still solve once a prefix is visible, and applying τ_{pre} as a decode bar would extend an illegal prefix that the decode rule is designed to stop.

Motivated by the gap, we perform continuation using only a short prefix of length t_0 , which has been enough to capture the difference between an illegal prefix and an answer-bearing one. Compared with decoding every admitted residual to D , the probe improves fidelity, since spending the full budget on an illegal prefix introduces additional decode work if a few tokens already suffice to govern the decision. In experiments, $t_0=128$ is the setting that returns MATH-500 any-finished accuracy at $k=16$ to the full fan-out. We do not assume testing prompts contain an answer mark at the first token.

2.5 In Front of a Decode-Time Host

ESC, speculative rejection, and DPTS remain the decode policy. PD-PRUNE only changes the set of residuals that reach them.

- ESC decodes an admitted residual out to D while its agreement window is open. On this mix the window stays open, so an admitted residual runs to $D=512$ and a residual refused by (2) is never decoded.
- Speculative rejection scores at horizon $\tau=256$ and drops the lower fraction $\alpha=0.5$ of the admitted set. The memory-limit trigger from that paper does not fire at these widths, so 256 is the cut.
- DPTS generates a mini-step of 100 tokens and early-stops a child whose confidence is below the decode bar.

The prefill bar is not retuned per host. $(\tau_{\text{pre}}, \tau_{\text{dec}})$ is the same pair on all three. When the prefill bar is raised to τ_{dec} , the admitted set is the live half. Those residuals are not the ones the host would have cut, the decode-side drop count falls to zero, and the three hosts converge to the same

decode length. The host remains complementary to PD-PRUNE, since our approach does not modify the host’s scoring rule.

3 Related Work

Agentic inference. Tree-of-Thoughts proposes candidate thoughts and then values them (Yao et al., 2023a). Self-consistency draws multiple chains and votes (Wang et al., 2023). ReAct interleaves reasoning with tools (Yao et al., 2023b). ESC closes the sample early once a window of answers agrees (Li et al., 2024). Speculative rejection spends a partial-reward horizon and then drops the lower quantile (Sun et al., 2024). DPTS grows a tree in mini-steps and stops a child whose confidence is low (Ding et al., 2025). In each case the score is a function of generated tokens, so an earlier threshold inside that family shortens decode and leaves prefill unchanged. The success of those cuts relies on tokens already being resident, i.e., it happens when the stack fails to skip abortable text but still stops it after a prefix. PD-PRUNE reads the residual before those tokens exist and leaves the host’s cut in place for the residuals it admits. The decode score is also not a token-draft acceptance test (Leviathan et al., 2023): speculative decoding checks a draft continuation of the same sequence, whereas the probe checks whether a sibling residual should continue to D .

Serving. Orca batches at iteration granularity (Yu et al., 2022). PagedAttention and automatic prefix caching address KV as blocks and reuse a block after its hash is stored (Kwon et al., 2023); SGLang’s radix cache is the same contract for program-structured calls (Zheng et al., 2024). DistServe and Splitwise place prefill and decode on separate instances and ship KV across the cut (Zhong et al., 2024; Patel et al., 2024). Those designs remove re-computation of a prefix that has already been stored, and they isolate time-to-first-token from inter-token latency. The first fan-out of a new trunk has nothing stored, and the connector has no opinion about a residual the search will abort. Training-time methods that change the model, and test-time workflows that add extra model calls per response, are orthogonal: they require a larger compute budget than a pair of string scores. PD-PRUNE is the admission step in front of that kernel and that transfer. It is complementary to the existing serving stack, while not modifying the prompt or introducing extra test-time overhead in response processing. Section 4

measures the gates. It does not propose a new page allocator.

4 Experiments

4.1 Experiment Setup

We evaluate PD-PRUNE on Qwen3-8B at temperature 0 with prefix caching and tensor parallelism 1, on four A100 40GB GPUs. By default we use the ordered pair in Equation 6 and probe length $t_0=128$. We evaluate with GSM8K under three decode-time hosts and with MATH-500 under the probe rule. We also report peak KV, quality, and offered load on a serving stack whose live occupancy is the committed spine, using six model families. Please refer to Appendix A for host parameters and metric definitions. Note that PD-PRUNE is complementary to the hosts, since our approach does not modify their scoring rules or require additional test-time model calls.

GSM8K hosts. Sixteen GSM8K problems. Each problem is a fan-out of $k \in \{4, 8, 16\}$ residuals and a decode budget $D=512$. Each fan-out is a controlled mix: half the residuals are live strategy instructions, and half alternate a repeated loop with illegal text. Index 0 is always a live strategy. *Any-finished* counts a problem if any admitted branch produces the reference number. *Winner* counts a problem if the branch selected as the winner does. Both are reported over 16. ESC, speculative rejection, and DPTS are the decode hosts. The host alone submits every residual. PD-PRUNE applies τ_{pre} first and leaves the host’s cut in place.

MATH-500 probe. The first 32 MATH-500 problems at level 4 or higher, $k \in \{4, 8, 16\}$, $D=1024$. Half the residuals are contest strategies and half are loops or illegal text. The decode bar is τ_{dec} . Full fan-out decodes every residual to D . Discard drops every residual below that bar before prefill. Probe admits those residuals for t_0 tokens and extends only when the generated prefix clears τ_{dec} .

4.2 Result Analysis

We summarize the results in Table 1 and Figure 3. PD-PRUNE significantly reduces the prefill shared by three decode-time hosts, while maintaining any-finished accuracy.

Specifically, with the prefill gate off, the three hosts share one prefill of 5008 tokens. Turning on τ_{pre} drops the loops, leaves illegal text, and cuts that prefill to 3848 on every host. ESC moves the

$k=16$	Prefill	Host	$+\tau_{\text{pre}}$	Any
ESC	5008 $\rightarrow$ 3848	17.05 s	14.13 s	16/16
SR	5008 $\rightarrow$ 3848	13.14 s	12.47 s	16/16
DPTS	5008 $\rightarrow$ 3848	11.86 s	11.49 s	16/16

Table 1: Prefill pruning at τ_{pre} in front of three hosts. Any-finished stays 16/16. The prefill column is identical across hosts because the gate, not the host cut, changes W , as predicted by Proposition 1.

most, because its baseline decodes the loops out to D . Speculative rejection and DPTS move less, because they were already stopping the loop and illegal half at their own cut. The gate removes the prefill of the loop, which none of those cuts avoids. This highlights a deficiency of pruning only after tokens exist: the host can stop an abortable residual, but it cannot unspend W .

Figure 3(a) raises the prefill bar from τ_{pre} to τ_{dec} , which also drops illegal text. Any-finished holds at $k=16$. End-to-end time converges across the three hosts, and the host’s later cut fires on nobody: the prefill gate has already removed every residual that cut would have rejected.

Table 2 separates the two gates. Decode pruning alone never changes prefill, and the time ranking is the ranking of the cut. Prefill pruning alone, with no host cut, is already faster than ESC at τ_{pre} and faster than all three hosts when the prefill bar is τ_{dec} . Stacking a host on τ_{pre} still stops some of the admitted residuals. Stacking a host on τ_{dec} does not, because the admitted set is the live half. Any-finished holds throughout. Winner accuracy is not identical to any-finished: dropping illegal residuals can change which branch the host ranks first. The gate preserves a correct branch in the admitted set. Appendix E reports $k=4$ and $k=8$, where the higher prefill bar does drop any-finished accuracy.

The two scores of a loop or an illegal residual are not interchangeable, which is the empirical content of the score gap in Section 2.4. On the residual, illegal text lies above τ_{pre} . On a generated prefix that is still illegal, the decode score lies below τ_{dec} . Applying τ_{dec} as a prefill bar therefore drops illegal text, which (2) at τ_{pre} would have admitted. On MATH-500 at large k that is the discard row of Table 3. Applying τ_{pre} as a decode bar would do the opposite: an illegal prefix still clears τ_{pre} , so the probe would extend a residual the decode rule is designed to stop.

Table 3 and Figure 3(b) compare the three prefill policies on MATH-500. Discarding every residual

below τ_{dec} is the fastest curve and the least accurate. The probe spends t_0 tokens on the uncertain residuals, then extends a residual only when its prefix clears τ_{dec} . At $k=16$ any-finished returns to the full fan-out. At $k=4$ and $k=8$ it does not. Recovery is a wide-fan-out result. Appendix F sweeps $t_0 \in \{64, 128\}$, loop-skipping, and a decode-only short budget. The $k=16$ match to full accuracy is specific to $t_0=128$ among the probe lengths we compare; $t_0=64$ reaches 18/32.

The prefill gate does not lengthen the committed spine. Appendix K measures that spine against recomputation and automatic prefix caching. Peak KV stays below the cache on six model families, because live occupancy is the spine. On Qwen3-8B the pruned system keeps GSM8K quality with that cache and lowers end-to-end time. Under a 1 s P99 time-to-first-token line, the shorter spine admits a higher offered load.

Table 4 holds the batch width fixed and compares two admission rules at τ_{dec} . Ignoring the score and keeping one residual drops any-finished accuracy on both hosts. Keeping every residual at or above τ_{dec} restores it. The savings in Table 1 come from dropping loops, and at τ_{dec} from dropping illegal text as well. They do not come from collapsing the fan-out onto its winner.

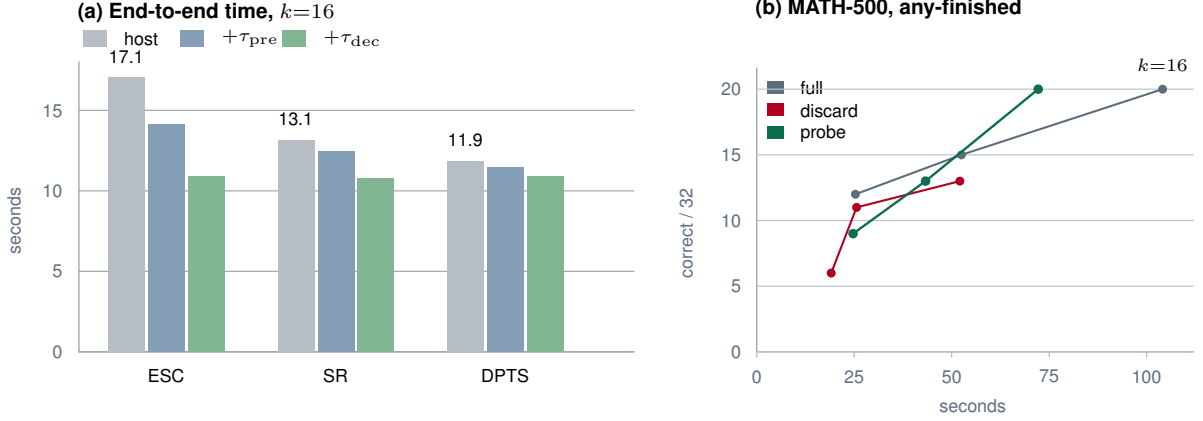


Figure 3: Qwen3-8B. (a) GSM8K controlled mix, $k=16$, $n=16$, $D=512$. Each group is one host. The gray bar is the host alone, blue adds τ_{pre} , and green sets the prefill bar to τ_{dec} . Any-finished accuracy holds on every bar. At the higher prefill bar the three hosts meet. (b) MATH-500, $n=32$, $D=1024$. Each polyline connects $k=4$, $k=8$, and $k=16$. Discarding residuals below τ_{dec} before prefill is fast and less accurate. The probe under (τ_{pre}, τ_{dec}) meets the full fan-out at $k=16$.

Setting	Admit	Cut later	Prefill	Decode	Time	Any	Winner
<i>Decode pruning only.</i> No τ_{pre} . Every residual is prefilled.							
ESC, budget $D=512$	16	0	5008	131072	17.05 s	16/16	11/16
SR, horizon 256	16	8	5008	98304	13.14 s	16/16	12/16
DPTS, mini-step 100	16	8	5008	78336	11.86 s	16/16	12/16
<i>Prefill pruning only.</i> Decode runs each admitted residual to $D=512$.							
τ_{pre}	12	0	3848	98304	14.13 s	16/16	11/16
τ_{dec} as prefill	8	0	2768	65536	10.92 s	16/16	10/16
<i>Both.</i> The same prefill bar, then the host cut on what remains.							
SR + τ_{pre}	12	4	3848	81920	12.47 s	16/16	10/16
DPTS + τ_{pre}	12	4	3848	71936	11.49 s	16/16	12/16
SR + τ_{dec}	8	0	2768	65536	10.80 s	16/16	10/16
DPTS + τ_{dec}	8	0	2768	65536	10.90 s	16/16	10/16

Table 2: Ablation at $k=16$ on the GSM8K controlled mix. “Cut later” is the number of admitted residuals the host stops at its own horizon. Any-finished is unchanged. Winner accuracy moves by at most two problems. When the prefill bar is τ_{dec} , the host cut fires on nobody and the decode columns meet.

	$k=4$	$k=8$	$k=16$
Full	12/32, 25.3 s	15/32, 52.5 s	20/32, 104.1 s
Discard	6/32, 19.1 s	11/32, 25.6 s	13/32, 52.1 s
Probe	9/32, 24.7 s	13/32, 43.3 s	20/32, 72.2 s

Table 3: MATH-500, Qwen3-8B, $n=32$, $D=1024$. Each cell is any-finished accuracy and end-to-end time. At $k=16$ the probe matches the full fan-out and is 31% faster. At $k=4$ and $k=8$ it does not fully recover accuracy.

$k=16$, bar τ_{dec}	Admit	Prefill	ESC	SR
Keep one, ignore score	1	414	11/16	9/16
Keep all above τ_{dec}	8	2768	16/16	16/16

Table 4: Winner-only admission on the GSM8K mix. A single reserved slot drops any-finished accuracy on both hosts. Scoring, and keeping the whole live half, restores 16/16.

5 Discussion

We presented a lightweight approach to efficient agentic inference. By identifying and pruning abortable residuals during KV-cache encoding of the residual, and by continuing an uncertain residual only when a short prefix clears a second bar, our approach ensures that prefill and transfer are spent on live strategies rather than on text the search will abort. Experiments show that PD-PRUNE exhibits a host-invariant reduction in shared prefill, significantly lowering end-to-end time while preserving large-fan-out any-finished accuracy on GSM8K and MATH-500. Collapsing either decision onto the other bar, or onto a single reserved slot, is what loses problems. Our work highlights a practical split for enhancing the efficiency of LLMs in agentic serving, complementary to decode-time hosts, prefix caches, and prefill-decode disaggregation.

6 Limitations

The scores are surface tests: repetition, an illegal-math pattern, an answer mark, and character statistics. They are not a learned reward model, and a residual that is wrong without being repetitive passes the prefill gate. The evaluation mix is constructed so that half the fan-out is live strategy text and half is a loop or illegal string. It is a controlled test of the two thresholds, not a trace of a production agent. Any-finished accuracy can stay flat while winner accuracy moves, and at $k=4$ even τ_{pre} drops one GSM8K problem. The MATH-500 probe recovers full any-finished accuracy at $k=16$ and does not do so at $k=4$ or $k=8$. The pruning study uses one model and temperature 0, so the tables are single-run measurements. The connector numbers in Appendix H are an accounting model, not a measured transfer time. We do not explore training-based alternatives such as finetuning a pruner, since we target the setting without a heavy compute budget.

References

Yifu Ding, Wentao Jiang, Shunyu Liu, Yongcheng Jing, Jinyang Guo, Yingjie Wang, Jing Zhang, Zeng-mao Wang, Ziwei Liu, Bo Du, Xianglong Liu, and Dacheng Tao. 2025. Dynamic parallel tree search for efficient LLM reasoning. In *ACL*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Effi-

cient memory management for large language model serving with PagedAttention. In *OSDI*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *ICML*.

Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. 2024. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. In *ICLR*.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*.

Hanshi Sun, Momin Haider, Ruiqi Zhang, Huitao Yang, Jiahao Qiu, Ming Yin, Mengdi Wang, Peter Bartlett, and Andrea Zanette. 2024. Fast best-of-N decoding via speculative rejection. In *NeurIPS*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *ICLR*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023b. ReAct: Synergizing reasoning and acting in language models. In *ICLR*.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-based generative models. In *OSDI*.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient execution of structured language model programs. In *NeurIPS*.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *OSDI*.

A Additional Details

Metrics. We evaluate GSM8K with two accuracy numbers and two cost numbers. Any-finished $\uparrow$ is

the fraction of problems for which any admitted residual produces the reference number. Winner $\uparrow$ is the fraction for which the residual selected as the winner does. Prefill $\downarrow$ is the measured GPU prefill token total. Time $\downarrow$ is end-to-end wall-clock seconds on four A100 40GB GPUs. For MATH-500 we report any-finished and time on the first 32 problems at level 4 or higher. Peak KV is the maximum live bytes of key-value cache; offered load is the concurrent request rate under a 1 s P99 time-to-first-token line.

Hosts. ESC uses an agreement window of length 5. On the controlled mix the window does not fill, so an admitted residual runs to $D=512$. Speculative rejection uses horizon 256 and dropped fraction 0.5, applied to the admitted set. DPTS uses a mini-step of 100 tokens. The memory-limit early trigger from speculative rejection does not activate at $k \leq 16$ on this hardware.

Admission rules. The default rule keeps every residual at or above τ_{pre} , including index 0. Winner-only keeps a single reserved slot and ignores the score. Both are measured at a fixed batch width in Table 4. PD-PRUNE is implemented on top of the host-alone baseline. It is complementary to the hosts, since it does not modify their scoring rules.

Controlled mix. For each problem, half of the residuals are live strategy instructions and half alternate a repeated loop with illegal text. Index 0 is always live. The mix is a controlled test of the two bars, not a production trace. Worked strings are in Appendix J.

B Score Functions

The runs in this paper use one ordered pair,

$$(\tau_{\text{pre}}, \tau_{\text{dec}}) = (0.15, 0.45). \quad (6)$$

A loop scores 0.02 on the residual and 0.04 on a prefix. Illegal text scores 0.22 on the residual and 0.16 on a prefix. An answer mark scores 0.92. Thus τ_{pre} sits between a loop and illegal text, and τ_{dec} sits between an illegal prefix and an answer mark. Raising the prefill bar to τ_{dec} is the faster operating point reported in the tables.

Both scorers are deterministic functions of a string. They do not call the model. Table 5 evaluates the residual kinds in the controlled mix.

Text	Reason	Prefill	Decode
Repeated <code>*loop*</code> span	loop	0.02	0.04
undefined <code>nan ...</code>	illegal	0.22	0.16
Live strategy instruction	ok	≈ 1.00	—
Prefix containing <code>####</code> or <code>\boxed</code>	answer	—	0.92
Empty string	empty	0.00	0.00

Table 5: Scores on the residual kinds in the evaluation mix. A dash means that scorer is not applied to that string. The decode column is the score of a generated prefix of that kind, not a second pass over the residual.

Prefill. After the loop and illegal tests, (4) mixes an alphanumeric fraction with character uniqueness. The coefficients 0.55 and 0.45 are fixed. The measurements score the residual text.

Decode. The answer-mark test runs before the illegal test, so a prefix that both looks noisy and contains an answer mark is an answer (0.92). The numeric mix in (5) adds digit and operator mass so that a healthy math prefix can continue before an answer mark appears. The planner prior that decides who is probed is

$$0.35 \min(1, 2\alpha) + 0.65 \min(1, 8\eta)$$

on the residual, with loop 0.04 and illegal 0.16. It ranks residuals for the host. It is not substituted for s^{pre} .

Early prefix inside prefill. A separate test, not used by the tables in Section 4, inspects the first fraction $\alpha=0.20$ of a residual. It fires when that slice scores below τ_{pre} for a reason other than ordinary text. The reported measurements use the full-residual loop test rather than this partial abort.

C Proof of Proposition 1

Proposition (Host-invariant prefill). *Let $\text{Keep} = \{i : \text{keep}_i\}$ be the admitted set of (2). For any decode-time host that scores only generated tokens, the prefill work W equals the measured prefill of Keep and does not depend on the host.*

Proof. A decode-time host is a map from generated prefixes to a stop/continue decision (Li et al., 2024; Sun et al., 2024; Ding et al., 2025). By (1), a prefix $y_{i,t}$ exists only after the KV cache H_i of z_i has been encoded. Therefore the host cannot remove z_i from the prefill batch: its cut acts on decode length, not on whether ρ_i is queued. Equation (2) is computed from residual text before any kernel, so Keep is independent of the host. W is defined

as the tokens the kernel runs on that set. Hence W is a function of Keep alone. The three hosts in Table 1 therefore share one prefill column at each bar, and differ only in decode-to-cut. $\square$

The proposition does not claim that end-to-end time is host-invariant. Time still ranks by the cut: ESC to D , speculative rejection to horizon 256, DPTS to a mini-step of 100. Prefill pruning changes the time ranking only by changing Keep, which is the empirical content of Table 2.

D Token Accounting

The decode columns of Table 2 are exact products of the admission counts, the cut, $D=512$, and $n=16$ problems. Write a for the number of admitted residuals per problem and c for how many of those the host stops at horizon h , with the rest running to D . Decode tokens are $n(c \cdot h + (a - c) \cdot D)$.

- ESC, no prefill gate: $a=16$, $c=0$, h unused. $16 \cdot 16 \cdot 512 = 131072$.
- Speculative rejection: $a=16$, $c=8$, $h=256$. $16 \cdot (8 \cdot 256 + 8 \cdot 512) = 98304$.
- DPTS: $a=16$, $c=8$, $h=100$. $16 \cdot (8 \cdot 100 + 8 \cdot 512) = 78336$.
- Prefill at τ_{pre} only: $a=12$, $c=0$. $16 \cdot 12 \cdot 512 = 98304$.
- Prefill at τ_{dec} only: $a=8$, $c=0$. $16 \cdot 8 \cdot 512 = 65536$.
- SR + τ_{pre} : $a=12$, $c=4$, $h=256$. $16 \cdot (4 \cdot 256 + 8 \cdot 512) = 81920$.
- DPTS + τ_{pre} : $a=12$, $c=4$, $h=100$. $16 \cdot (4 \cdot 100 + 8 \cdot 512) = 71936$.
- Either host + τ_{dec} : $a=8$, $c=0$. 65536 .

The mix explains the admission counts. For $k=16$, eight residuals are live and eight are not. That half alternates loop and illegal text, four of each. τ_{pre} drops the four loops and keeps the four illegal residuals: $8 + 4 = 12$. Raising the prefill bar to τ_{dec} drops both kinds: 8. Index 0 is among the live eight, so the forced keep does not add a further slot. At $k=8$ the same halving admits 6 at τ_{pre} and 4 at τ_{dec} . At $k=4$ it admits 3 and 2.

Prefill tokens are measured, not assigned by this product, because prefix caching makes the incremental prefill depend on the token ids of each residual. The measured totals at $k=16$ are 5008, 3848,

and 2768 for admission 16, 12, and 8. The same totals appear for every host at a given τ_{pre} , which is the empirical content of Figure 1(b) and of Proposition 1.

E Fan-out Width

ESC	Base	+ τ_{pre}	Δ	Any	Winner	Prefill
$k=4$	8.65 s	8.22 s	-5%	16→15	9→9	1320→1018
$k=8$	10.84 s	10.19 s	-6%	16→16	11→9	2624→2020
$k=16$	17.05 s	14.13 s	-17%	16→16	11→11	5008→3848

Table 6: ESC across fan-out width at the default prefill bar. The relative saving grows with k because more loop residuals are refused. At $k=4$ any-finished drops by one problem. At $k=8$ any-finished holds and the winner moves by two.

	Prefill	Admit	ESC	Any
$k=16$ base	5008	16	17.05 s	16/16
$k=16$, τ_{pre}	3848	12	14.13 s	16/16
$k=16$, τ_{dec} as prefill	2768	8	10.92 s	16/16
$k=8$, τ_{dec} as prefill	1448	4	—	14/16
$k=4$, τ_{dec} as prefill	732	2	—	14/16

Table 7: Why the default prefill bar is τ_{pre} . At $k=16$, raising it to τ_{dec} is faster and any-finished still holds. At $k=8$ and $k=4$ the same bar admits only the live half, which is four and two residuals, and any-finished falls to 14/16. Speculative rejection at $k=16$ with that higher bar is 10.80 s and DPTS is 10.90 s, both 16/16.

The $k=4$ row of Table 6 is the reason Section 6 does not describe τ_{pre} as accuracy-neutral at every width. One problem that the full ESC fan-out solved is absent once the single loop residual at $k=4$ is refused. We keep τ_{pre} because raising it to τ_{dec} , which looks better on the $k=16$ bars of Figure 3(a), fails the same test two widths down (Table 7).

F Probe Sweep

Two comparisons in Table 8 fix the design of Algorithm 1. Dropping loops and then decoding everyone else to D is the prefill gate with no probe. At $k=16$ it is slightly faster than the probe (71.4 s versus 72.2 s) and four problems less accurate. The accuracy belongs to the residuals that fail the prefill-side live test and still solve after a prefix is visible. A short decode that never consults s^{dec} reaches 17/32: the extension test, not the mere existence of a short budget, is what returns the last problems. Including loops in the probe matches 20/32 at 74.4 s. We still skip loops before the kernel. Their prefill score is below τ_{pre} , the probe does not buy accuracy at $k=16$, and it spends the extra 2.2 s.

Setting	$k=4$	$k=8$	$k=16$
Full fan-out	12/32, 25.3 s	15/32, 52.5 s	20/32, 104.1 s
Discard every residual below τ_{dec} before prefill	6/32, 19.1 s	11/32, 25.6 s	13/32, 52.1 s
Drop loops only, decode the rest to D	11/32, 23.0 s	17/32, 41.5 s	16/32, 71.4 s
Short-decode the low scores, no extension test	10/32, 23.1 s	15/32, 42.3 s	17/32, 74.0 s
Probe $t_0=64$, extend iff prefix $\geq \tau_{\text{dec}}$	8/32, 24.0 s	16/32, 42.0 s	18/32, 72.5 s
Probe $t_0=128$, extend iff prefix $\geq \tau_{\text{dec}}$	9/32, 24.7 s	13/32, 43.3 s	20/32, 72.2 s
Probe loops as well ($t_0=128$)	10/32, 24.9 s	15/32, 43.3 s	20/32, 74.4 s

Table 8: MATH-500 ablation. Each cell is any-finished accuracy and wall-clock time. Dropping loops only is fast and, at $k=16$, loses four problems relative to the full fan-out (16/32 versus 20/32). Probing the non-loop residuals for 128 tokens and then applying τ_{dec} is the only row that matches 20/32. Probing the loops too matches accuracy and spends 2.2 s more. At $k=8$, several rows land within two problems of the full fan-out; we do not treat a one-problem difference on $n=32$ as a ranking.

At $k=16$ and $t_0=128$, every non-loop uncertain residual is probed and then extended, and decode tokens fall from 524288 to 393216. The $k=8$ probe at $t_0=64$ reaches 16/32, one above the full fan-out’s 15/32. That is within the one-problem noise we decline to interpret; the claim we use in the body is the $k=16$ tie with the full fan-out at lower time.

G Host Cuts

The three hosts are unchanged except for the set (2) hands them.

ESC. The published stopping rule waits until a window of agreeing samples is full. The window length used here is 5. The controlled fan-out does not fill that window: some branches are loops or illegal text, and the live strategies do not agree early. Every admitted residual is therefore decoded to D . This is why ESC’s baseline is the most expensive row in Table 2, and why refusing loops before prefill moves ESC more than it moves DPTS.

Speculative rejection. The partial-reward horizon is $\tau=256$ tokens and the dropped fraction is $\alpha=0.5$, applied to the admitted set rather than to the original k . With 12 admitted residuals the host stops 4; with 8, all eight are above the mass the rule would reject, and the later-cut column is 0. The memory-limit early trigger described in that paper does not activate at $k \leq 16$ on this hardware, so we do not take credit for it.

DPTS. Each admitted child generates a mini-step of 100 tokens. A child whose confidence is below τ_{dec} stops; the others continue to D . The mini-step is the earliest of the three cuts, which is why DPTS is the fastest host when the prefill gate is off (11.86 s) and why adding the gate moves it by only 0.37 s at τ_{pre} .

Order inside one problem. The prefill gate runs before any decode kernel for that fan-out. The host then runs on the children that were admitted. Refused residuals are absent from the decode batch. The gates are ordered by cost: a text test, then prefill, then a short probe or the host horizon, then the rest of D . Only admitted KV is eligible to cross to the decode instance.

H Connector Accounting

The seconds in Section 4 are measured GPU time. The calculation below is a separate accounting model of the transfer term X .

Consider 10 sessions, $k=4$, trunk $L=256$, residual length $\ell=32$, with one of the four residuals a loop. A disaggregated prefill that has no admission computes every residual and ships every page:

$$W = X = 10 \cdot 4 \cdot (256 + 32) = 11520$$

tokens. Charging $12 \mu\text{s}$ per prefill token and $2 \mu\text{s}$ per transferred token, plus 0.8 ms to abort a refused child, gives a fan-out of 162.1 ms of which 23.0 ms is transfer. Refusing the loop before insert leaves three residuals, each still carrying its trunk under this model:

$$W = X = 10 \cdot 3 \cdot 288 = 8640.$$

Transfer falls to 17.3 ms and the modeled fan-out to 121.8 ms. The 2880 tokens removed are one residual per session, trunk included, because this model replicates the trunk on the connector. A page cache that hits the trunk would shrink both the baseline and the pruned column by the same shared prefix; the residual term would remain. The model shows that the prefill gate and the connector payload move together once admission precedes the transfer. The measurements already include prefix caching, so the prefill column in Section 4 is incremental, not this 11520-token construction.

I Winner-Only Admission

Table 4 uses batch size 8 on the $k=16$ GSM8K mix at bar τ_{dec} . The score-blind rule fills one slot. Measured prefill is 414 tokens and any-finished accuracy is 11/16 under ESC and 9/16 under speculative rejection. The score rule admits the eight live residuals (2768 prefill tokens) and both hosts return to 16/16. The gap is the live strategies other than index 0. Several problems are solved by a non-winner live residual and by none of the single reserved slots the blind rule happened to keep. Keeping the argmax of the prefill score would be a third policy; we do not claim the prefill score ranks answer correctness, only that thresholding it is a different decision from keeping one branch.

J Worked Residuals

The live half cycles through six instructions:

1. Translate the story into equations, then solve for the missing value.
2. Work backwards from the asked quantity to the given numbers.
3. Name each intermediate quantity in order and add them last.
4. Build a small table of given facts, then apply one operation at a time.
5. Estimate the magnitude first, then do exact arithmetic to confirm.
6. Convert all units, cancel common factors, then compute.

None of them matches the loop or illegal patterns, and (4) scores them at 1.0. The other half is the alternating pair

- a repeated `*loop*` span, scored 0.02 because a span of length 8 to 40 repeats at least three extra times;
- the string “undefined nan junk residual that cannot be a proof”, scored 0.22 by the illegal-math pattern.

A generated prefix is scored only after it exists. The illegal decode score and the answer score in Figure 2 are prefix scores. They are not a second labeling of the residual.

K Memory and Offered Load

The prefill gate does not lengthen the committed spine. Figures 4, 5, and 6 measure that spine against recomputation and automatic prefix caching on the same serving stack. Live occupancy is $b(L + \ell_*)$ after abort, so peak KV stays below automatic prefix caching.

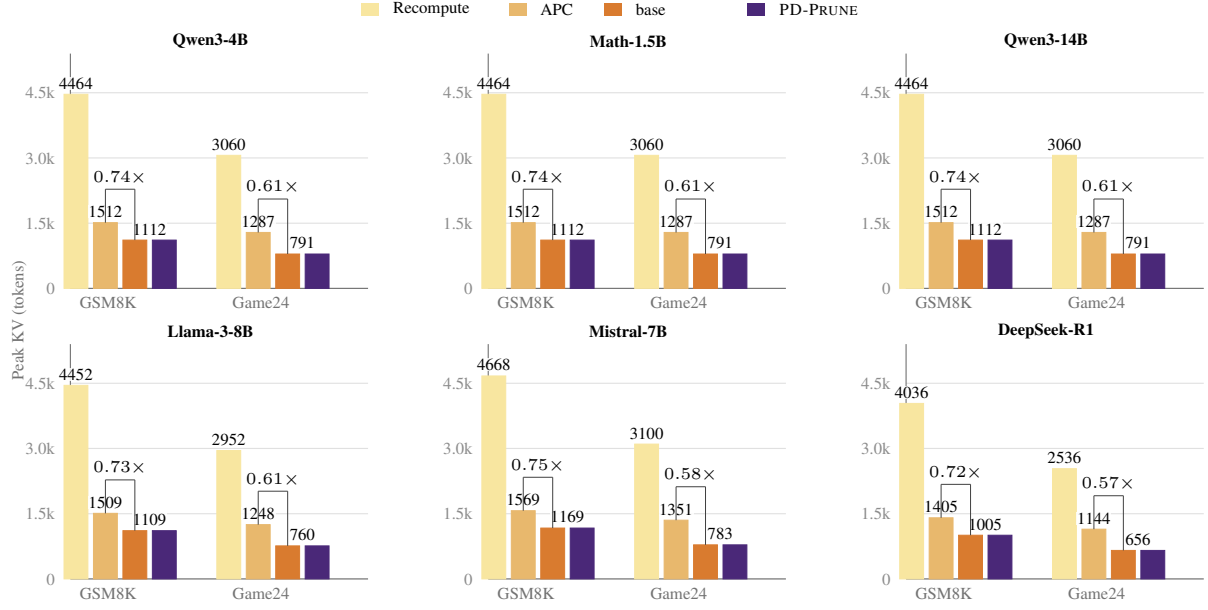


Figure 4: Peak KV on six models ($n=40$, $tp=2$). Each panel is one family; groups are GSM8K and Game24. Bars are recompute, APC, the unpruned spine, and PD-PRUNE. PD-PRUNE shares the spine with the unpruned runtime, so the two right-hand bars match. The annotated ratio is spine over APC.

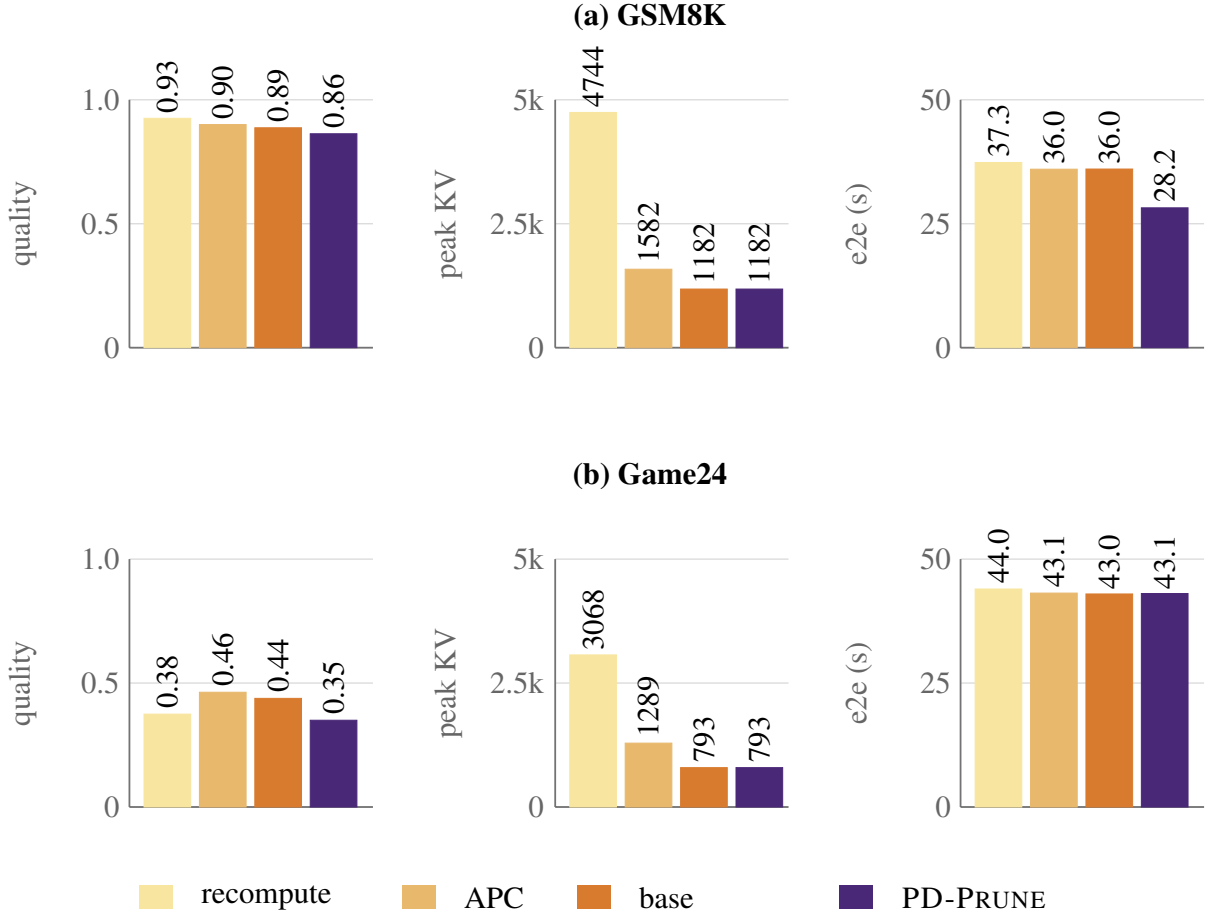


Figure 5: Four systems on Qwen3-8B ($n=80$, $tp=2$, $D=512$). Each panel has its own axis. Quality is higher-better; peak KV and end-to-end time are measured values, so a shorter bar is better. (a) GSM8K. (b) Game24. PD-PRUNE lowers peak KV below APC on both workloads. GSM8K quality stays with APC, and PD-PRUNE also lowers GSM8K end-to-end time.

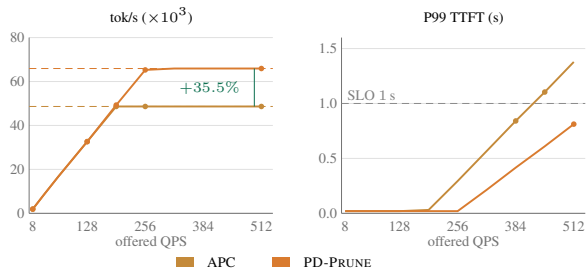


Figure 6: Offered-load sweep from the measured DeepSeek-R1 GSM8K peaks. Left: saturated tokens per second plateau at concurrency times the decode rate. Right: APC crosses the 1 s P99 line first; PD-PRUNE stays under it at a higher QPS.